CyClaw-Net-Viewer

How it works

Why Rust, how the Mac .app launches, what the code is doing, and which dependencies can bite later.

Read this after the User Guide. That one is clicks and colors. This one is for reviewing the source in this repository with two altitudes: ELI5, then Tech 101 with real file and crate names.

1. ELI5

Your Mac already keeps a list of every phone line (TCP) and every walkie-talkie channel (UDP) that programs have open. CyClaw-Net-Viewer is a live scoreboard. About once a second it asks the operating system: who is talking, on which port, to whom, and which app owns that socket? Then it draws a table and colors the rows when something new appears, changes, or hangs up.

Ping is a postcard (ICMP). This app does not list postcards. It only lists phone lines. That is why pinging google.com did not light up red. curl or the real CyClaw process would, because those open TCP.

Orange means “this line goes off the machine.” Green/blue/yellow/red are the Sysinternals-style events (new out, new in, state change, gone). The Finder icon is just a folder named .app with the compiled program inside. Double-click runs that program. There is no Python hiding in the bundle.

2. Why Rust instead of Python

Python would have been the fast way to parse lsof in a terminal. It was the wrong shape for a double-click desktop table that walks every process every second and must not inherit CyClaw’s own dependency graph.

Need	Rust here	Python would have been
Ship to a Mac	One Mach-O binary plus Info.plist. No interpreter on the target.	py2app / PyInstaller: a Python runtime, site-packages, and a fat .app. Easy to break Gatekeeper and PATH.
1 Hz snapshot of all PIDs	Direct libproc FFI. No GIL. No shelling out to lsof and parsing columns.	subprocess + parse, or ctypes to the same C APIs with more glue and worse packing bugs.
Watch CyClaw (a Python app)	Different language and crate tree. This viewer does not import OpenTelemetry, httpx, or CyClaw extras.	Same ecosystem. Easy to accidentally pull a telemetry SDK into the watcher.
Reproducible compiler	rust-toolchain.toml pins rustc 1.85.0.	Whatever python3 is on PATH, plus pip drift.
Universal binary	Two cargo --target builds and lipo.	Two Pythons or a universal CPython embed; messy.
Cost we paid	eframe/icu MSRV pins; netstat2 bindgen at compile time; more files than a 40-line script.	Faster first prototype if the only goal was “print lsof in a window.”

Bottom line: Python is excellent for CyClaw itself and for one-off checks. Rust is why this viewer is a ~6 MB native .app you can leave running at 1 Hz without dragging a second copy of the observed stack along for the ride.

3. What makes it a standalone Mac app

Not Tauri, not Electron, not SwiftUI, not py2app. Four pieces stacked:

3.1 Compile a native binary

rustc 1.85.0 builds crate netboard into an executable named netboard. scripts/make-app.sh sets MACOSX_DEPLOYMENT_TARGET=12.0 so the Mach-O's minos is 12.0 (Monterey). It builds aarch64-apple-darwin and, when the SDK allows, x86_64-apple-darwin, then lipo -create so one file is Intel and Apple Silicon.

3.2 Draw a window with eframe (egui)

eframe 0.31.1 is a small Rust GUI framework. egui is immediate-mode UI (every frame the code says "here is a table"). winit talks to AppKit for the window. glow is OpenGL for pixels. That is why you get a real macOS window without a WebView and without bundling Chromium.

3.3 Wrap it in a .app bundle

macOS does not launch a raw binary from Finder the way you want for a product. A .app is a directory. Ours is:

```
CyClaw-Net-Viewer.app/
  Contents/
    Info.plist   # name, bundle id local.cyclaw.netviewer, min OS 12.0
    MacOS/netboard  # the lipo'd executable
```

scripts/make-app.sh copies the binary, copies Info.plist, then ad-hoc signs: codesign --force --sign -. That is a local signature with no Developer ID and no Apple notarization. Gatekeeper's first-open rule is Right-click → Open. There is no App Sandbox. Sandboxing would hide other processes' sockets and defeat the tool.

Double-click = launchd/Finder exec of Contents/MacOS/netboard. No Python, no Node, no JVM.

4. Tech 101 — what the code is doing

All of this lives under src/. Binary entry is main.rs. Library modules are re-exported from lib.rs.

4.1 The loop

Step	Who	What
1. Kernel	Darwin libproc	Every process's file descriptors, including sockets, with local/remote and TCP state.
2. snapshot.rs	netstat2 + libc	Turn those FDs into Endpoint structs: pid, proto, addrs, process name, path, In/Out/Listen.
3. diff.rs	pure Rust	Compare to last tick. New / changed / gone (gone lingers two ticks).
4. dns.rs	8 worker threads	PTR getnameinfo, then a forward A lookup so a v6 row can still show hostname (IPv4).
5. app.rs	eframe table	Paint rows. Orange if off-box and no event color. Snapshot runs on a background thread.

4.2 Files to read, in order

File	Job
main.rs	If argv has --cli, print a snapshot and exit. Else app::run() opens the window.

File	Job
snapshot.rs	get_sockets_info(IPv4|IPv6, TCP|UDP). Map netstat2 TCP states onto our enum. proc_name / proc_pidpath per PID (cached per tick). Direction: LISTEN → Listen, SYN_SENT → Out, else In if that PID also listens on the local port. is_offbox: not unspecified, not loopback (v4-mapped loopback counts as local).
diff.rs	Identity key = (pid, proto, ip version, local, remote). Missing key → NewIn or NewOut. Same key, different TCP state → Changed. Missing from next snapshot → Deleted linger=2, then drop. Returning after delete is New, not Changed.
app.rs	eframe::run_native("CyClaw-Net-Viewer"). A thread calls snapshot+diff, stores Vec<Row> in a mutex (poison → into_inner; panic → catch_unwind and keep last good rows). The UI thread paints TableBuilder, legend, Off-box only filter. Close Connection is kill.rs SIGTERM after a modal.
dns.rs	Resolver::request queues IPs. Workers call getnameinfo(NI_NAMEREQD), then to_socket_addrs for IPv4. UI shows hostname (1.2.3.4):port when both exist.
cli.rs / kill.rs	Tcpvcon-style flags -a -c -n. terminate() refuses pid 0 and our own pid.

Off-box orange is a paint overlay, not a fifth diff event. Highlight::None plus is_offbox(remote) plus Color remotes → orange. New/changed/gone still win.

5. Dependencies that could be a risk later

Runtime is the Mach-O plus macOS libproc and OpenGL. No Python, no Node, no telemetry SDK in this process. The risks are mostly compile-time pins and the GUI stack's appetite for newer rustc.

Piece	Role	Risk	Why it can hurt later
eframe / egui / winit / objc2 / glow	Window and table	**High**	New egui releases raise MSRV. We already had to pin icu/idna/image/writeable so rustc 1.85 still builds. Apple AppKit bindings churn. Do not cargo update eframe casually.
netstat2 0.11	Socket list via libproc	**Med**	Uses bindgen + libclang at compile time (needs Xcode CLT). Quiet crate. UDP remotes are missing. If xnu struct layouts change, bindgen may still compile and return garbage.
Pinned icu_* / idna_adapter / url / image / writeable	MSRV shims for eframe	**Med**	They exist only so 1.85 compiles. A blanket cargo update drops the pins and the build dies. Leave the exact versions in Cargo.toml.
webbrowser (egui-winit links)	Unused "open URL" helper	**Low**	We do not open links, but the feature pulled the icu/idna stack. Harmless at runtime; annoying at compile pin time.
arboard	Copy line / copy remote	**Low**	Clipboard. Small. Could lag behind objc2 versions if eframe jumps.
libc	proc_name, proc_pidpath, getnameinfo, kill	**Low**	Thin FFI. Darwin symbols have been stable for years. Wrong struct packing would be our bug, not libc's.
Ad-hoc codesign, no notarization	Finder launch	**Med**	Gatekeeper and future macOS releases can get stricter with unsigned apps. Right-click → Open works today; it is not a promise.
No App Sandbox	See other processes' sockets	**Low (by design)**	Do not "harden" by enabling the sandbox. That would empty the table. The risk is App Store rules, which this app is not aiming at.

Build machine only: clang/libclang for netstat2's bindgen. The .app you copy to another Mac does not include clang, Rust, or Python.

Practical rule

- Do not run cargo update without checking rustc 1.85.0 still builds.
- Do not bump eframe past 0.31.x while the toolchain is 1.85.0.
- If netstat2 bitrots, replace it with a small hand-written libproc walker (the Darwin headers are in the macOS SDK).
- Never add an HTTP client, OpenTelemetry, or “just one analytics call” to this binary. It exists to watch those things, not to be one.

6. What we deliberately did not use

- Tauri / Electron / a local web page — extra Chromium and a second language.
- PyInstaller / py2app — see section 2.
- Godot — the gamedev skill’s Mac ship bar applied; the engine did not.
- Network Extension / pcap / PKTAP — overkill for a netstat table; needs entitlements.
- True TCB close — Darwin has no SetTcpEntry. SIGTERM is the honest substitute.

User guide: docs/CyClaw-Net-Viewer-User-Guide.pdf. This document: docs/CyClaw-Net-Viewer-How-It-Works.pdf.